PRACTICAL AI AND DATA ENGINEERING · 2026

AI Engineering in 2026 â€” A Practical 20-Part Course

Audience: a developer with basic Python. **Outcome:** build, evaluate, secure, deploy, and operate industry-grade AI products. **Pace:** 32 weeks, 8â€“12 hours/week. Every part follows the same practical pattern: explain, diagram, example, code, production design, cost/token note, interview prompts, project, and resources.

How to work through it

Build one portfolio repository per project. Every repository needs a README, locked dependencies, tests, `.env.example`, an evaluation dataset, an architecture diagram, and a cost/latency report. Start with managed APIs and simple workflows; introduce a framework, agents, or GPUs only when a measured requirement needs them.

Part 1 â€” AI Foundations (Weeks 1â€“2)

Concepts. AI is software that performs tasks associated with perception, language, decisions, or learning. A short history: symbolic/rule-based systems â† statistical ML â† deep learning â† transformer foundation models â† tool-using multimodal systems. **Narrow AI** solves bounded tasks; **AGI** is a hypothetical general capability across domains; **superintelligence** is speculative capability beyond humans. Do not present either AGI or superintelligence as a deployed product category. ML learns from examples; deep learning is ML using multi-layer neural networks; reinforcement learning (RL) learns actions from rewards. **Terminology.** Dataset, feature, label, parameter, training, inference, loss, epoch, batch, overfitting, generalization, benchmark, latency, throughput, hallucination, grounding, evaluation, model card, and drift.

```
examples + objective -> training -> model parameters -> inference(input) -> prediction/action

                                    ^                                 |

                                    +---- evaluation and feedback ------+
```

Real-world use. Rules handle a known shipping fee; ML predicts churn; an LLM explains the prediction and drafts retention outreach. This combination is more reliable than asking a chatbot to do all three. **Python.**

```
python
def rule_shipping(country: str) -> float:

    return 0.0 if country == "US" else 15.0
```

Production / economics. Keep deterministic business rules outside the model. Measure a business metric (resolved tickets, conversion, error rate), not only model fluency. Classical ML is usually lower-latency and lower-cost than an LLM for a stable tabular prediction. **Pitfalls / interview.** â€œAIâ€ is not synonymous with LLM. Ask: What is the difference between training and inference? Why does overfitting harm a production system? **Project.** Build a small support-triage API: deterministic routing plus a trainable priority classifier. **Resources.** [Google ML rules](https://developers.google.com/machine-learning/guides/rules-of-ml), [scikit-learn guide](https://scikit-learn.org/stable/user_guide.html).

Part 2 â€” Practical Mathematics (Weeks 3â€“4)

Concepts. Learn only the mathematics needed to reason about models.

Linear algebra: a vector is an ordered list; a matrix transforms vectors; dot product measures alignment; norms measure size. Embeddings use vectors and cosine similarity.

Probability: uncertainty as numbers from 0â€“1; conditional probability asks â€œgiven evidenceâ€□; distributions describe possible outcomes.

Statistics: mean/median, variance, sampling, correlation, confidence intervals, and train/test splits prevent fooled-by-noise decisions.

Calculus: a derivative tells local change; the chain rule lets gradients flow through neural-network layers.

Gradient descent / optimization: repeatedly move parameters opposite the loss gradient. Learning rate trades slow progress for instability; regularization reduces overfitting.

```
input vector --[weights + bias]--> prediction --compare to label--> loss

     ^                                                    |

     +---------------- gradient descent updates weights -------+
```

Real-world use. Rank the most relevant policy paragraph by cosine similarity, then show a user the source. **Python.**

```
python
import numpy as np

def cosine(a, b): return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

loss = lambda w: (w - 3) ** 2

w, lr = 0.0, 0.1

for _ in range(30): w -= lr * (2 * (w - 3))

print(w, loss(w))
```

Production / economics. Normalize vectors consistently; compute embeddings once and reuse them. Use confidence intervals before claiming an A/B-test win. Do not hand-derive backpropagation in application work, but understand what an optimizer is optimizing. **Pitfalls / interview.** Correlation is not

causation; a biased sample produces a biased estimate. Ask: Why can a high learning rate diverge? Why is cosine similarity useful for embeddings? **Project.** Implement linear regression and gradient descent from NumPy, then compare against scikit-learn. **Resources.** [3Blue1Brown linear algebra](https://www.3blue1brown.com/topics/linear-algebra), [StatQuest](https://www.youtube.com/@statquest).

Part 3 â€” Machine Learning (Weeks 5â€“7)

Concepts. **Supervised learning** maps inputs to labels: classification predicts a category; regression predicts a number. **Unsupervised learning** finds structure without labels: clustering, anomaly detection, dimensionality reduction. **RL** learns a policy by reward through interaction. Use supervised ML for churn/fraud/forecasting; use unsupervised methods to segment customers or inspect data; use RL only where sequential rewards and safe simulation/feedback exist. **Algorithms.** Linear/logistic regression: transparent baseline. Decision trees/random forests: strong tabular baseline. **XGBoost:** boosted trees, often excellent for tabular data but needs tuning. **SVM:** effective in moderate-size high-dimensional data. **KNN:** simple local similarity baseline but slow at scale. K-means: simple clustering; DBSCAN: irregular-density clusters.

```
versioned data -> features -> train candidates -> validation -> registry -> API/batch prediction

                                    |                       |            |

                               metrics + bias checks ------ monitor drift --+
```

Python.

```
python
from sklearn.ensemble import RandomForestClassifier

X, y = [[0, 1], [1, 1], [0, 0], [1, 0]], [0, 1, 0, 1]

model = RandomForestClassifier(random_state=7).fit(X, y)

print(model.predict([[1, 1]]))
```

Production / economics. Establish a simple baseline; split by time if the future resembles production; version features/data/model. Trees on CPUs often cost much less than LLM calls and are easier to explain. Monitor calibration, drift, false-positive cost, and business impact. **Pitfalls / interview.** Never leak future data. Ask: Random forest vs XGBoost? Precision vs recall? When is clustering inappropriate? **Project.** Churn predictor comparing logistic regression, random forest, and XGBoost; publish confusion matrices and a threshold decision. **Resources.** [scikit-learn](https://scikit-learn.org/stable/user_guide.html), [XGBoost](https://xgboost.readthedocs.io/).

Part 4 â€” Deep Learning (Weeks 8â€“10)

Concepts. Neural networks learn layered representations via backpropagation. **PyTorch** is common in research and flexible production work; **TensorFlow/Keras** remains common in established enterprises and mobile/TF-serving ecosystems. **CNNs** exploit image locality. **RNNs/LSTMs** process sequences but are now often replaced by transformers. A **transformer** uses attention: each token weighs relevant tokens. Positional encoding supplies order because attention alone has none.

```
tokens/images -> encoder layers (attention + MLP + residuals) -> task head -> loss -> gradients
```

Python.

```
python
import torch

net = torch.nn.Sequential(torch.nn.Linear(2, 8), torch.nn.ReLU(), torch.nn.Linear(8, 1))

x = torch.tensor([[1.0, 2.0]])

print(net(x))
```

Production / economics. Use transfer learning and pretrained checkpoints; train on GPUs only when the workload justifies it. Store checkpoints, dataset versions, seeds, and metrics. GPU time, data labeling, and experimentation are the real costs; do not train a model when a hosted model or classical baseline meets the requirement. **Pitfalls / interview.** More layers/data do not guarantee generalization. Ask: Why residual connections? Why is attention quadratic in naÃ¯ve sequence length? CNN vs transformer for images? **Project.** PyTorch image classifier with augmentation, validation, error analysis, and an exported inference API; reproduce one small Keras model to understand both ecosystems. **Resources.** [PyTorch tutorials](https://pytorch.org/tutorials/), [TensorFlow guides](https://www.tensorflow.org/learn), [Attention paper](https://arxiv.org/abs/1706.03762).

Part 5 â€” Large Language Models (Weeks 11â€“13)

Concepts. An LLM is a transformer trained to predict tokens at scale, then adapted to follow instructions. A tokenizer maps text to token IDs from a fixed **vocabulary**. The **context window** bounds input plus generated output. Temperature, top-p, and top-k change sampling diversityâ€”not factuality. Training commonly progresses through pretraining, supervised instruction tuning, and preference optimization such as **RLHF** or **DPO**. **Adaptation / efficiency.** Fine-tuning modifies behavior; **PEFT**, **LoRA**, and **QLoRA** train small adapters rather than all weights. **MoE** activates a subset of expert parameters per token. Quantization stores weights at fewer bits for cheaper/faster inference; distillation transfers behavior from a stronger teacher to a smaller student.

```
raw text -> tokenize -> pretrain next-token prediction -> instruction/preference tuning -> deploy

user prompt + context ----------------------------------------> decode tokens -> response
```

Python.

```
python
def sample(logits, temperature=0.7, top_p=0.9):

    if temperature <= 0: raise ValueError("use deterministic decoding separately")

    return "provider tokenizer/sampler performs this operation"
```

Production / economics. Set explicit max output, temperature, timeout, retry, and model version. Fine-tune only after a prompt/RAG baseline and a held-out evaluation set. Quantization may reduce GPU memory but can hurt quality and tool-call reliability; validate it on your task. **Pitfalls / interview.** Context is not long-term memory. DPO is not simply â€œRLHF without quality work.â€ Ask: LoRA vs full fine-tuning? What makes MoE cheap per token but operationally complex? How do top-p and temperature differ? **Project.** Token-budget inspector plus a controlled experiment: zero-shot vs few-shot vs retrieval vs LoRA plan on one structured-extraction task. **Resources.** [Hugging Face LLM course](https://huggingface.co/learn/llm-course/chapter1/1), [PEFT](https://huggingface.co/docs/peft), [DPO paper](https://arxiv.org/abs/2305.18290).

Part 6 â€” Generative-AI Model Families (Week 14)

Concepts. ChatGPT/OpenAI, Claude/Anthropic, Gemini/Google, DeepSeek, Mistral, Llama/Meta, and Qwen/Alibaba are model families and platforms, not interchangeable checkboxes. Closed providers commonly offer fast managed inference and integrated tools; open-weight families offer deployment control and customization. Capabilities, prices, context limits, and terms change frequentlyâ€”benchmark the current versions against your workload.

```
application -> internal model gateway -> policy/route -> provider or self-hosted model -> validated result
```

Selection guide. Use the model that wins your task evaluation at acceptable latency/cost/privacy. Startups tend to choose managed APIs for speed. Enterprises may add a gateway, regional routing, approved vendors, audit controls, or self-hosted Llama/Qwen/Mistral/DeepSeek-class models when privacy, customization, or sustained volume justify it. **Python.**

```
python
def route(task, risk):

    return "strong-model" if risk == "high" else ("small-model" if task == "classify" else "balanced-model")
```

Production / economics. Keep provider-specific SDK code behind one interface; preserve test fixtures; evaluate fallback models before an outage. Never hardcode price claims or model names in product logic. Cost is input tokens + output tokens + tool/retrieval/hosting/engineering overhead. **Pitfalls / interview.** Leaderboard scores do not equal your quality. Ask: How would you compare providers fairly? What does vendor lock-in look like operationally? **Project.** Build a model-router benchmark with 50 representative tasks, quality rubric, p95 latency, token usage, and cost per successful outcome. **Resources.** Official model docs: [OpenAI](https://platform.openai.com/docs/), [Anthropic](https://docs.anthropic.com/), [Google](https://ai.google.dev/), [Hugging Face](https://huggingface.co/docs).

Part 7 â€” Prompt Engineering (Week 15)

Concepts. A prompt is an interface contract: role/instructions, task, trusted context, examples, constraints, and output schema. **Zero-shot** uses instructions only; **few-shot** adds examples. Chain-of-thought can improve some reasoning but should not be treated as a trusted audit record; request a concise answer and verifiable evidence. **ReAct** alternates reasoning/action observations. XML labels can separate trusted sections; JSON/schema outputs make machines safer. System/developer/user messages set different instruction layers; application policy must not rely only on prompt wording.

```
system/developer policy + user task + retrieved context -> model -> JSON schema validation -> UI/tool
```

Python.

```
python
from pydantic import BaseModel

class Triage(BaseModel): category: str; urgency: int

prompt = "Return JSON matching: category string, urgency integer 1-5. Ticket: {{text}}"
```

Production / economics. Prefer structured outputs/function calling to prose parsing. Version prompts; add evals before edits; keep examples short and representative; place stable prefixes consistently for supported prompt caches. XML/Markdown/JSON do not magically save tokensâ€”the right format is the clearest, shortest format for the model and task. **Pitfalls / interview.** Prompts cannot authorize money movement or bypass RBAC. Ask: Zero-shot vs few-shot? Why validate model JSON again? How do you test a prompt regression? **Project.** Create an extraction service with structured output, prompt versions, 30 golden tests, and an injection-resistance test suite. **Resources.** [OpenAI prompting] (https://platform.openai.com/docs/guides/prompt-engineering), [Anthropic prompting] (https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview).

Part 8 â€” AI Agents (Weeks 16â€“17)

Concepts. An agent is a bounded loop that uses a model to select tools/steps toward a goal. Agentic AI is useful when paths vary; a deterministic workflow is better when steps are known. Single-agent systems are easier to debug; multi-agent systems are justified only for genuinely separable roles. Planning decomposes work; reflection checks results but can add latency/cost. Short-term memory is current state; long-term memory is explicitly saved, permissioned facts. Retrieval fetches knowledge. Orchestration persists state, budgets, retries, and approvals. MCP is a protocol for connecting models to external tools/data; treat every MCP server as third-party code with permissions.

```
state -> planner -> approved tool -> observation -> evaluator/budget -> next step or approval

  ^                                                                    |

  +-------------------------- checkpoint / resume ---------------------+
```

Frameworks. Plain Python first. LangGraph for durable state graphs; CrewAI/AutoGen for higher-level multi-agent prototypes; PydanticAI for typed Python agents; OpenAI Agents SDK for its provider-aligned agent primitives. Choose based on required control, observability, and team familiarityâ€”not hype. **Python.**

```
python
ALLOWED = {"lookup_order"}

def execute_tool(name, args):

    if name not in ALLOWED: raise PermissionError("tool not permitted")

    return {"status": "shipped", **args}
```

Production / economics. Typed schemas, least privilege, per-run time/token/tool budgets, idempotency keys, durable checkpoints, human approval for writes, trace replay, and a deterministic fallback. Multi-agent coordination multiplies tokens, latency, and failure modes. **Pitfalls / interview.** Never grant an agent unrestricted shell, SQL, or browser permissions. Ask: Agent vs workflow? How do you stop loops? How do you resume after a crash? **Project.** Build an order-support agent that reads status, drafts a refund request, pauses for approval, and records every action. **Resources.** [MCP](https://modelcontextprotocol.io/), [LangGraph](https://docs.langchain.com/oss/python/langgraph/overview), [PydanticAI](https://ai.pydantic.dev/), [OpenAI Agents SDK](https://openai.github.io/openai-agents-python/).

Part 9 â€” RAG (Weeks 18â€“19)

Concepts. Retrieval-augmented generation (RAG) fetches current, private, authorized evidence before generating. Embeddings encode semantic similarity; a vector database stores/searches them. Chunking chooses meaningful passages; chunk size/overlap balance recall against irrelevant context. Metadata contains source, version, tenant, ACL, time, and type. Hybrid search combines lexical and vector signals; reranking reorders candidates; citations show evidence.

```
documents -> parse/clean -> chunks + metadata -> embed -> vector/keyword index

question -> ACL filter -> hybrid retrieve -> rerank -> grounded prompt -> answer + citations
```

Databases. Pinecone: managed vector service; Weaviate/Qdrant/Milvus: vector platforms with managed/self-host options; Chroma: convenient local/prototype option. Postgres + pgvector is often the best first production choice when scale/features are modest and relational metadata is central. Benchmark recall, filters, operations, and price against your corpus. **Python.**

```
python
def chunks(text, size=400, overlap=60):

    step = size - overlap

    return [text[i:i+size] for i in range(0, len(text), step)]
```

Production / economics. Enforce ACLs before retrieval; retain source/version; tombstone deleted docs; evaluate retrieval separately from answer quality; abstain if evidence is weak. Embed incrementally by content hash, retrieve few strong chunks, deduplicate overlap, and cache frequent results. **Pitfalls / interview.** A vector similarity score is not truth. Ask: Why hybrid search? How do you prevent cross-tenant leakage? How do chunk choices affect faithfulness? **Project.** Citation-first PDF handbook bot with ingestion, tenant filters, hybrid retrieval, reranking, 100 eval questions, and a â€œnot foundâ€ response. **Resources.** [FAISS](https://faiss.ai/), [Sentence Transformers](https://www.sbert.net/), [RAG paper] (https://arxiv.org/abs/2005.11401), provider docs above.

Part 10 â€” AI Infrastructure (Weeks 20â€“21)

Concepts. GPUs accelerate parallel tensor operations; CUDA is NVIDIAâ€™s programming/runtime platform. Inference generates predictions/tokens; serving packages it behind an API. vLLM improves LLM serving throughput with efficient KV-cache management; Ollama makes local model experimentation simple; TensorRT optimizes NVIDIA inference. Docker packages software; Kubernetes schedules/scales containers; FastAPI exposes typed Python services.

```
client -> WAF/API gateway -> FastAPI -> queue/model gateway -> GPU inference pods (vLLM/TensorRT)

                             |                  |                |

                       Postgres/Redis        tracing          object/vector store
```

Python.

```
python
from fastapi import FastAPI


app = FastAPI()


@app.get("/healthz")


def healthz(): return {"ok": True}
```

Production / economics. Start managed/serverless for spiky workloads. Self-host when privacy, custom models, or sustained utilization produces a measured win. Add queues for long work, health checks, autoscaling, GPU utilization metrics, quotas, and fallback capacity. Idle GPUs are expensive; batch only where it does not violate latency SLOs. **Pitfalls / interview.** Donâ€™t serve lengthy generation synchronously without cancellation/timeouts. Ask: What is continuous batching? Why separate API servers from workers? CUDA vs a model framework? **Project.** Containerize a FastAPI RAG service; add asynchronous ingestion and deploy a staging stack with health checks and load test. **Resources.** [vLLM](https://docs.vllm.ai/), [Ollama](https://docs.ollama.com/), [TensorRT](https://docs.nvidia.com/deeplearning/tensorrt/), [Kubernetes](https://kubernetes.io/docs/home/).

Part 11 â€” Cost Optimization (Week 22)

Concept. Optimize cost per successful business outcome, preserving evaluated quality and safetyâ€”not simply token count.

```
request -> classify complexity -> cache? -> route small/large model -> cap tools/context/output -> measure
outcome + spend
```

Techniques. Prompt/context compression removes repeated or irrelevant text. Exact caching returns the same result for an identical request; semantic caching reuses a validated answer for a sufficiently similar query and must include tenant/ACL/version in its key. Batch inference improves throughput for offline work. Model routing reserves stronger models for hard/high-risk tasks. Smaller models, quantization, embedding reuse, token reduction, and streaming can lower cost or perceived latency. Streaming usually does **not** lower token cost; it improves time-to-first-token and enables cancellation. **Python.**

```
python
from hashlib import sha256


cache = {}


def cached(prompt, model, version):


    return cache.get(sha256(f"{model}:{version}:{prompt}".encode()).hexdigest())
```

Production example. A support RAG system: cache permission-safe FAQs; use hybrid retrieval of 4 chunks; route classification to a small model; use a strong model only when the evaluator/complexity classifier requires it; set 900 output-token and 3-tool caps; batch nightly embedding; dashboard cost by tenant and resolved case. **Pitfalls / interview.** Do not share semantic-cache answers across users/tenants. Ask: Why is cost/request misleading? How do you test a router? When does self-hosting pay off? **Project.** Add budgets, tiered routing, cache hit metrics, batch embedding, and an outcome-based cost dashboard to the capstone. **Resources.** [OpenAI prompt caching] (https://platform.openai.com/docs/guides/prompt-caching), [vLLM](https://docs.vllm.ai/).

Part 12 â€” Token Optimization (Week 23)

Concepts. A token is a tokenizer-defined text chunk, not a word or character. Token counting must use the tokenizer of the target model. Markdown can reduce tokens when headings/lists replace repetitive prose, but tables/backticks may increase them. XML can make boundaries clear; JSON is compact and machine-parseableâ€”neither universally uses fewer tokens. Measure, do not guess.

```
raw conversation/docs -> token count -> prune irrelevant turns -> summarize/externalize artifacts ->
retrieve evidence -> capped prompt
```

Techniques. Use concise task instructions, short representative few-shot examples, schema instead of prose, context pruning, retrieval/reranking, conversation summaries, reference IDs, compression with evaluation, and output caps. Preserve source IDs/version/ACL after compression. Avoid silently compressing legal/medical/financial evidence. **Python.**

```
python
def rough_token_estimate(text: str) -> int:

    return max(1, len(text) // 4)  # planning estimate only; use target tokenizer in production
```

Production / economics. Enforce per-feature token budgets and log input/output tokens. A 20% context reduction may cut latency and cost, but only ship it if retrieval/answer evals stay within a predeclared regression budget. **Pitfalls / interview.** â€œMore contextâ€ can make answers worse through distraction. Ask: What is lost-in-the-middle? Why count output tokens separately? **Project.** Build a prompt linter that shows token composition, flags duplicate context, and compares original/compressed prompt evaluation scores. **Resources.** [tiktoken](https://github.com/openai/tiktoken), [Hugging Face tokenizers](https://huggingface.co/docs/tokenizers/).

Part 13 â€” Model Comparison (Week 24)

Method. Do not publish a static winner. Build a living scorecard for current GPT, Claude, Gemini, Llama, DeepSeek, Qwen, and Mistral versions using your prompts, regions, contracts, and dates. Compare speed (TTFT/p50/p95), task accuracy, coding tests, total cost, usable context, reasoning-task success, multimodal accuracy, structured/function-call reliability, privacy/deployment options, and outage behavior.

| Dimension | How to measure | Common mistake |

|---|---|---|

| Accuracy/reasoning | blinded labeled tasks + human calibration | using one impressive demo |

| Coding | unit tests, security checks, patch acceptance | judging only prose explanation |

| Speed | time-to-first-token and p95 end-to-end | measuring provider time only |

| Cost | input/output/tool/hosting cost per successful task | comparing price per million tokens alone |

| Context | useful recall/faithfulness at long input | assuming advertised window is useful context |

| Multimodal/tools | schema success + task correctness | testing only happy-path JSON |

Python.

```
python
def winner(rows):

    # quality threshold first; then select lowest measured cost among qualifiers

    return min((r for r in rows if r["quality"] >= .90), key=lambda r: r["cost"])
```

Production / economics. Maintain model-version fixtures and scheduled re-evals. Pin versions when available; canary upgrades; route by task/risk. Context/window, price, and availability are time-sensitive and must be rechecked in official docs before procurement. **Interview.** Design a fair model bake-off. Explain why a long context window does not eliminate RAG. **Project.** Publish a reproducible comparison harness and a decision recordâ€”not a permanent leaderboard. **Resources.** Official provider model documentation and your organizationâ€™s current pricing/contract pages.

Part 14 â€” AI Coding (Week 25)

Tools. VS Code + extensions, Cursor, Windsurf, Claude Code, Codex, and GitHub Copilot help inspect, generate, refactor, test, and review code. Their exact capabilities and privacy controls change, so use the current vendor docs and company policy.

```
ticket/spec -> plan -> AI-assisted small change -> local tests/lint/typecheck -> human review -> CI ->
deploy
```

Best workflow. Give the assistant relevant files and acceptance criteria; ask for a plan; make small diffs; run tests; inspect the diff/security implications; commit only validated work. Use repo instructions, test fixtures, linters, typed schemas, and CI as guardrails. Never paste secrets/customer data or accept generated dependency/code changes without review. **Python.**

```
python
def acceptance(result):


    return result["tests_pass"] and result["security_reviewed"] and result["diff_reviewed"]
```

Production / economics. Centralize policy, approve extensions/connectors, keep audit logs where required, and measure developer outcomes rather than autocomplete volume. AI coding reduces drafting time; it does not transfer ownership of correctness to a tool. **Interview.** How do you use coding agents safely? How would you prevent an agent from making a broad, risky refactor? **Project.** Use an AI coding tool to implement a feature through a written plan, test-first changes, review checklist, and before/after productivity note. **Resources.** Current official documentation for the selected IDE/agent; [OWASP secure coding](https://owasp.org/www-project-top-ten/).

Part 15 â€” MLOps and LLMOps (Weeks 26â€“27)

Concepts. MLOps/LLMOps brings CI/CD, deployment, evaluation, monitoring, observability, tracing, prompt/model versioning, and A/B testing to probabilistic systems. Traces link user request â†’ retrieval â†’ model/tool calls â†’ output â†’ feedback. Version code, prompt, model, data/index, evaluation set, and configuration together.

```
Git PR -> unit/integration/security/evals -> staging -> canary/A-B -> production

  ^                                                    |

  +---- traces + metrics + feedback + incidents + rollback -+
```

Python.

```
python
def release_gate(m):

    return m["groundedness"] >= .90 and m["p95_s"] <= 5 and m["cost_per_task"] <= .05
```

Production / economics. Use feature flags, canaries, release gates, redacted OpenTelemetry-compatible traces, SLO alerts, prompt registry, model registry, and rollback runbooks. A/B-test measurable outcomes with safeguards; never test harmful behavior merely to maximize engagement. **Pitfalls / interview.** Raw traces can leak PII. Ask: What belongs in a release gate? What differs between offline and online evals? How do you identify a bad model version? **Project.** Add CI eval gates, tracing, a dashboard, canary release simulation, and incident runbook to the RAG bot. **Resources.** [OpenTelemetry](https://opentelemetry.io/docs/), [MLflow](https://mlflow.org/docs/latest/), [LangSmith evaluation](https://docs.langchain.com/langsmith/evaluation).

Part 16 â€” AI Security (Week 28)

Threats. Prompt injection attempts to override instructions; jailbreaks seek prohibited behavior; data leakage exposes secrets/PII; unsafe tool use creates real-world harm. Guardrails are layers: authentication, RBAC/ABAC, scoped tools, input/output moderation, PII detection/redaction, secrets management, rate limits, network boundaries, audit logs, human approval, and incident response.

```
input -> auth + tenant/RBAC -> PII/injection checks -> model -> output checks -> approval -> scoped action

                    \----------------- immutable audit trail -----------------/
```

Python.

```
python
def can_refund(role, amount): return role == "finance_approver" and amount <= 500
```

Production / economics. Enforce authorization in application/data layers before retrieval and tool execution, not in model instructions. Use a secret managerâ€”not `.env` in production. Redaction lowers both risk and tokens. Security incidents and compliance remediation dwarf ordinary API costs. **Pitfalls / interview.** Do not expose arbitrary SQL/shell/browser tools. Ask: Explain indirect prompt injection. How do you protect multi-tenant RAG? Where are secrets stored? **Project.** Threat-model the capstone; add RBAC, tenant retrieval filters, redaction, injection corpus tests, tool allowlist, approvals, and audit records. **Resources.** [OWASP LLM Top 10](https://genai.owasp.org/llm-top-10/), [NIST AI RMF](https://www.nist.gov/itl/ai-risk-management-framework).

Part 17 â€” Building Production AI (Weeks 29â€“30)

Architecture. Design for scaling, latency, caching, retries, fallbacks, monitoring, logging, and cost from day one. A production system needs timeouts, circuit breakers, idempotency, queues, backpressure, multi-tenant isolation, retention/deletion controls, and an explicit failure UX.

```
UI -> CDN/WAF -> API -> auth/policy -> model gateway -> providers/self-hosted inference

                  |       |              |                        |

              Postgres  Redis/cache    RAG/tools/queue          metrics/traces/logs
```

Python.

```
python
def retryable(status): return status in {429, 500, 502, 503, 504}
```

Production example. A customer-support bot caches safe FAQs, queues document ingestion, uses a circuit breaker during provider errors, falls back to search + escalation rather than invented answers, records a trace ID, alerts on p95 latency/groundedness/cost, and enforces tenant budgets. **Cost/token optimization.** Cache validated results, route tasks, cap outputs, reuse embeddings, batch offline work, cancel abandoned streams, and attribute spend by feature/tenant/outcome. Add retry jitter and never blindly retry non-idempotent write tools. **Interview.** Draw an architecture for 10Ã— traffic; explain graceful degradation and provider-outage behavior. **Project.** Deploy the capstone to staging with load tests, SLOs, cost dashboard, failure injection, backup/restore test, and rollback playbook. **Resources.** [Google SRE book](https://sre.google/sre-book/table-of-contents/), [AWS Well-Architected](https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html).

Part 18 â€” Portfolio Projects (run throughout)

Build these progressively, not as disconnected demos:

AI Chatbot: structured chat, streaming, feedback, safety UX.

RAG bot / PDF chat: ingestion, citations, ACLs, hybrid retrieval, evals.

Resume analyzer: schema extraction, bias/privacy review, human decision support only.

Meeting assistant: transcription, summary/action schema, consent/retention controls.

SQL agent: read-only scoped database, semantic layer, query preview, approval for writes.

Coding agent: sandboxed repository tools, tests, diff review, no unrestricted shell.

Research agent: source provenance, claim extraction, citation validation, approval steps.

Email agent: retrieval + drafts, send approval, idempotent send/logging.

Voice assistant: ASR, tool flow, TTS, interruption/latency design, transcript privacy.

Customer-support bot: full capstone: routing, RAG, tools, escalation, SLOs, cost controls.

For each: diagram, Python service, production deployment, eval set, threat model, token/cost report, five interview questions, and a demo video.

Part 19 â€” Interview Preparation (Weeks 31â€“32)

Create a **500-question bank**, organizedâ€”not memorizedâ€”as: 50 foundations/math, 80 ML/deep learning, 100 LLM/prompting, 80 RAG, 70 agents/tools/MCP, 50 MLOps/infra, 40 security, and 30 behavioral/product questions. For every answer: define the concept, give a tradeoff, draw a system, state metrics, and describe a failure mode.

Representative questions.

Explain bias/variance and choose a churn metric.

Derive why attention needs positional information.

RAG vs fine-tuning vs a larger context window?

Design chunking/evaluation for legal PDFs.

How do you stop a tool-using agent from repeating a charge?

What belongs in an LLM release gate?

How would you migrate models without a quality regression?

Design a multi-tenant, cited support assistant at 100 requests/second.

Defend against indirect prompt injection in retrieved web content.

Reduce a $100k/month LLM bill without lowering successful resolution rate.

System-design practice. Draw data flow, trust boundaries, API/async paths, storage, caching, observability, SLOs, costs, testing, and rollout. **Coding practice:** Python data structures, APIs, async I/O, SQL, tests, parsing/validation, basic ML, and a small RAG/tool loop. **Project.** Record ten 30-minute mock interviews; write an answer rubric; revise one capstone design after each mock. **Resources.** [System Design Primer](https://github.com/donnemartin/system-design-primer), official docs and your own project postmortems.

Part 20 â€” Latest AI Trends (maintain quarterly)

Track, test, and add to the scorecard rather than repeating headlines:

AI agents, tool use, MCP, browser/computer-use: increasingly useful for bounded, approval-gated workflows; reliability and permission design remain the bottleneck.

Reasoning models: can improve difficult tasks but require task-specific evaluation, budget caps, and careful latency/cost management.

Vision, voice, and multimodal models: make document, speech, and visual workflows practical; privacy, consent, and evaluation expand with modalities.

Open-source/open-weight models: Llama, Qwen, Mistral, DeepSeek-class ecosystems expand deployment choice; licenses, benchmarks, hosting, and security must be checked per release.

AI IDEs and coding agents: shift engineering toward specification, test, review, and safe tool permissions.

SLMs, edge/on-device AI: reduce latency, bandwidth, cost, and data exposure for narrow tasks; constrained memory/quality needs an explicit fallback.

Robotics: combines perception, planning, control, simulation, and strict safety constraints; do not extrapolate chat-agent reliability to physical autonomy.

Trend evaluation loop.

```
new capability -> threat/privacy review -> small benchmark -> cost/latency/quality scorecard -> sandbox
pilot -> gated rollout
```

Resources. [Stanford AI Index](https://aiindex.stanford.edu/report/), [MCP specification](https://modelcontextprotocol.io/), [OpenAI docs](https://platform.openai.com/docs/), [Hugging Face](https://huggingface.co/docs). Recheck primary sources, prices, licenses, and provider policies each quarter.

Final roadmap

```
0-2 months: Parts 1-5    -> Python + ML + deep learning + LLM foundations

3-4 months: Parts 6-10   -> prompting + agents + RAG + infrastructure

5-6 months: Parts 11-17  -> efficiency + model selection + LLMOps + security + production

7-8 months: Parts 18-20  -> portfolio, system design, interviews, continuously updated trends
```

Choose a specialization after the capstone: **Applied LLM Engineer** (product/RAG/agents), **ML Platform/MLOps** (serving/data/reliability), **Research Engineer** (PyTorch/training/papers), or **Decision AI** (forecasting/ranking/experimentation). The durable industry skill is not memorizing toolsâ€”it is proving an AI system is useful, safe, measurable, maintainable, and economical.